

Recocido Simulado por Aceptación por Umbrales para resolver el Problema del Viajero

Carlos Cabrera Ramirez

September 19, 2026

Abstract

En este documento presentaremos una implementación hecha en Rust para la heurística de *recocido simulado* en su variante de *aceptación por umbrales* para instancias del problema del viajero (Travelling Salesman Problem) obtenidas de una base de datos con 1,092 ciudades alrededor del mundo, las cuales tienen 123,403 conexiones conocidas. Dado que nuestra gráfica de conexiones no es completa, definimos una función de costo la cual penaliza cualquier arista que no exista en nuestra gráfica original; de igual manera generamos un normalizador que toma en cuenta esto y nos permite identificar cualquier solución como válida cuando su costo es ≤ 1 .

La búsqueda explora el espacio de permutaciones a través del intercambio de dos vértices, el cambio en el costo de estos intercambios se evalúa en tiempo constante. Las soluciones a aceptar se obtienen en lotes definidos dinámicamente durante la experimentación, detectando el equilibrio térmico cuando el promedio del costo de las soluciones aceptadas empieza a decelerar su decrecimiento.

Nuestra implementación corre experimentos diferentes de manera paralela y se le puede suplir la opción para terminar cada corrida con un barrido, el cual consiste en buscar a través de todo el vecindario de la solución aceptada por algún vecino cuya función de costo evalúa menor que la solución aceptada.

Reportamos soluciones en instancias de 40 y 150 ciudades, obteniendo resultados satisfactorios donde evaluamos la función de costo de 150 ciudades a un mínimo de 0.1365 y de 40 ciudades a un mínimo de 0.2638.

1 Introducción

El problema del viajero (TSP) es uno de los problemas más estudiados en la optimización combinatoria: dado un conjunto de ciudades y las distancias entre ellas, se debe encontrar la ruta más corta que visita cada una de las ciudades exactamente una vez. El problema fue formalizado y resuelto para instancias de tamaño considerable desde los años 50 [1], y desde entonces se ha vuelto un caso de estudio recurrente en optimización combinatoria. Este problema es NP-duro, por ende, incluso para instancias de tamaño lo suficientemente moderado se vuelve impráctico el hecho de enumerar (y explorar!) las $n!$ posibles permutaciones, o resolverlo de manera óptima en un tiempo razonable. Por ello recurrimos a las heurísticas: algoritmos los cuales no nos pueden garantizar un óptimo, pero sí producen buenas soluciones en un rango razonable de tiempo.

Implementamos *recocido simulado* (*simulated annealing*), una metaheurística inspirada en la metalurgia, donde un material se calienta y después se deja enfriar lentamente para reducir sus defectos, propuesta originalmente por Kirkpatrick, Gelatt y Vecchi [4] e, independientemente, por Černý [5] aplicándola específicamente al TSP. Para este análogo, nuestro algoritmo acepta al inicio muchas soluciones que pueden ser peores y progresivamente se vuelve más estricto conforme la

temperatura es menor, por ende podemos escapar de óptimos locales al inicio y converger en una buena solución al final.

El lenguaje de programación usado es Rust, se leen tanto las ciudades como sus conexiones de una base de datos en SQLite. Nuestro código se organiza en dos estructuras principales: **Tour** la cual encapsula la estructura de la solución, siendo esto la solución aceptada, la manera de obtener un vecino, así como la función de costo; por otro lado tenemos **TravelSalesmanProblem** el cual contiene el bucle del recocido. Nuestra implementación de igual manera hace uso de múltiples hilos para paralelizar el cálculo de múltiples soluciones.

2 Problema

2.1 Input

Nuestra base de datos contiene la tabla **cities** con 1,092 ciudades y la tabla **connections** la cual relaciona dos ciudades entre si, esta última tiene 123,403 conexiones las cuales serán las únicas válidas al correr nuestro programa.

Sea C el conjunto de ciudades y $E \subseteq C \times C$ el conjunto de conexiones. La gráfica $G = (C, E)$ *no es completa*, ie existen pares de ciudades las cuales no están conectadas. gráfica

Una *instancia* de nuestro problema es un subconjunto ordenado $I = \{c_1, \dots, c_n\} \subseteq C$ de n ciudades, dadas como una lista separada por comas de los identificadores de las ciudades (en base de datos)

2.2 Distancia entre ciudades

La distancia natural entre dos ciudades u, v esta calculada por la formula de Harversine en una esfera de radio $R = 6\,373\,000$ m:

$$h(u, v) = \sin^2\left(\frac{\varphi_v - \varphi_u}{2}\right) + \cos \varphi_u \cos \varphi_v \sin^2\left(\frac{\lambda_v - \lambda_u}{2}\right), \quad (1)$$

$$d(u, v) = 2R \cdot \text{atan2}\left(\sqrt{h(u, v)}, \sqrt{1 - h(u, v)}\right), \quad (2)$$

donde φ y λ son la latitud y longitud en radianes.

Para no depender en los valores de la tabla **connections**, nuestra implementación calcula todas las distancias con esta fórmula.

2.3 Solución y Función de Costo

Una solución es una permutación de $S = (s_1, \dots, s_n)$ de las ciudades de la instancia obtenida. Como nuestra gráfica no es completa, es posible que una permutación pueda hacer uso de una arista que no existe, para no descartar un sin fin de posibles soluciones, en vez de descartar la solución lo que haremos será asignarle un peso extremadamente alto, el cual no permitirá considerarlo nunca una solución factible. Para ello definimos la *máxima distancia* de la instancia,

$$d_{\max} = \max\{d(u, v) : (u, v) \in E, u, v \in I\}, \quad (3)$$

así como la *función de peso aumentada*

$$w'(u, v) = \begin{cases} d(u, v) & \text{si } (u, v) \in E, \\ d(u, v) \cdot d_{\max} & \text{de otra manera.} \end{cases} \quad (4)$$

Multiplicando por $d_{\max}$ nos garantiza que una arista faltante es mucho más costosa que cualquier arista existente, y dado que el factor por el que multiplicamos es la distancia natural, por ende cualesquiera dos aristas que no pertenezcan a nuestra gráfica serán comparables en tamaño.

Para hacer el costo independiente del tamaño de la instancia, definimos el *normalizador*: la suma de las $n - 1$ distancias más grandes (para aristas existentes) en la instancia,

$$N(I) = \sum_{k=1}^{n-1} \ell_k, \quad \ell_1 \geq \ell_2 \geq \dots \text{ son las distancias en } \{d(u, v) : (u, v) \in E, u, v \in I\}. \quad (5)$$

Finalmente, la función de costo está definida por

$$f(S) = \frac{1}{N(I)} \sum_{i=1}^{n-1} w'(s_i, s_{i+1}). \quad (6)$$

Cualquier permutación que use solo aristas existentes en nuestra gráfica tendrá 1 como cota superior del costo (dado que usa $n - 1$ vertices, en el peor de los casos siendo los $n - 1$ más grandes); cualquier permutación que haga uso de una arista inexistente tendrá un costo mayor a 1. Por ende podemos decir que cualquier permutación cuyo costo es menor o igual a 1 es *factible*, nuestro objetivo es encontrar la permutación que minimice este costo.

3 Heurística

3.1 Recocido Simulado

El recocido simulado es una heurística de búsqueda local que mantiene una sola solución en memoria. A partir de nuestra solución s se puede generar un *vecindario* s' al intercambiar una arista. Si s' es mejor que s esta primera será aceptada, si es peor será aceptada dependiendo en que tanto empeore ésta con respecto a nuestra solución, ie $f(s') < f(s) + T$ donde T es la temperatura actual del sistema. Esta temperatura iniciará en un valor arbitrario alto y se reducirá al multiplicarse por un factor de enfriamiento $\phi \in (0, 1)$, haciendo que peores soluciones se acepten en menor medida mientras más avancemos en la ejecución de nuestra heurística. solución

3.2 Aceptación por Umbrales

La variante del recocido simulado que implementamos aquí usará la técnica de *aceptación por umbrales* (*threshold accepting*), propuesta por Dueck y Scheuer [6] como una alternativa determinística al criterio probabilístico de Metropolis usado en el recocido simulado clásico. En esta usamos una regla determinística: un vecino s' será aceptado sí y solo sí

$$f(s') \leq f(s) + T. \quad (7)$$

Por ende, la temperatura incide directamente en el número de aceptadas, así como el umbral de "empeoramiento" el cual vemos como aceptable. Podremos observar que para valores altos de T nuestra búsqueda local actúa de manera muy similar a una caminata aleatoria, dado que es muy sencillo moverse a través del vecindario de soluciones – y aún más cuando nuestra solución ya es factible. En el momento en el que $T \rightarrow 0$ empezaremos a aceptar solo soluciones que mejoren a partir de la solución actual, esta fase es la que llamamos "explotación".

3.3 Lotes y Equilibrio térmico

Para cada temperatura procesaremos un *lote*: obtendremos una sucesión de vecions de los cuales aceptaremos un número L de ellos (o hasta que lleguemos a un número arbitrario de ejecuciones en el lote a procesar para evitar bucles infinitos en temperaturas muy bajas). Nuestro lote regresará el costo promedio de las soluciones aceptadas. Dentro de la ejecución de cada lote la temperatura se mantendrá constante siempre y cuando este promedio se mantenga decreciente entre lotes consecutivos, esto es lo que llamamos *equilibrio térmico*; en el momento en el que este promedio deje de decrecer multiplicaremos a T por nuestro factor de enfriamiento ϕ y repetiremos el proceso hasta que T caiga por debajo de una constante ε o cuando un lote deje de aceptar soluciones.

Algorithm 1 ObtenerLote

Input: temperatura T , solución actual s , tamaño de lote L

Output: costo promedio de las soluciones aceptadas y solución aceptada

```
1:  $c \leftarrow 0, r \leftarrow 0, i \leftarrow 0$ 
2: while  $c < L$  and  $i < 100L$  do
3:    $s' \leftarrow \text{VECINO}(s)$ 
4:   if  $f(s') \leq f(s) + T$  then
5:      $s \leftarrow s', c \leftarrow c + 1, r \leftarrow r + f(s)$ 
6:     if  $f(s) < f(s^*)$  then
7:        $s^* \leftarrow s$  ▷ mejor solucion global
8:     end if
9:   end if
10:   $i \leftarrow i + 1$ 
11: end while
12: return  $r/c, s$ 
```

Algorithm 2 AceptaUmbrales

Input: temperatura inicial T , factor ϕ, ε, L , solución inicial s

```
1:  $p \leftarrow 0$ 
2: while  $T > \varepsilon$  do
3:    $q \leftarrow \infty$ 
4:   while  $p \leq q$  do
5:      $q \leftarrow p$ 
6:      $p, s \leftarrow \text{OBTENERLOTE}(T, s, L)$ 
7:     if no hay solución aceptada en el lote then
8:       return  $s^*$ 
9:     end if
10:  end while
11:   $T \leftarrow \phi T$ 
12: end while
13: return  $s^*$ 
```

3.4 Vecindario

Un vecino a nuestra solución se obtiene a través de tomar dos vértices distintos de manera estocástica e intercambiando las ciudades que estos representan. Este vecindario es simple, conecta todo el espacio de búsqueda y además nos permite calcular un cambio en el costo en tiempo constante.

3.5 Barrido

Una vez que nuestro recocido termina, de manera opcional podemos *barrer* el vecindario de nuestra solución: buscamos en todo el vecindario de nuestra solución aceptada algún vecino que mejore (por poco que sea) la función de costo, si llegamos a encontrar tal vecino, lo tomamos como nueva solución y repetimos este proceso hasta ya no encontrar ninguno. Este paso nos permitirá llegar al óptimo local del vecindario en caso de que no nos encontremos ahí.

4 Diseño de la Solución

4.1 Código

El proyecto esta separado en módulos los cuales implementan tanto el I/O como los modelos del proyecto:

- `io/args`: parser para la línea de comandos, usando `clap`.
- `io/instance_reader`: nos permite leer la instancia a resolver del TSP. Permite leer desde un archivo o directamente como una cadena en la consola de comandos.
- `io/instance_writer`: persiste los resultados y el reporte de cada corrida del experimento.
- `models/db`: conector a SQLite asi como los modelos donde se vacía esta información.
- `models/city`, `models/connection`: entidades de base de datos. `City` implementa la distancia de Haversine
- `models/solution`: la estructura del `Tour`, se encarga de construir los parámetros con los que trabajara el problema (matriz de peso, normalizador), así como se encarga de los estados actuales del problema (solución actual, su peso y provee la operación para obtener un vecino)
- `models/problem`: la estructura de `TravelSalesmanProblem`, tiene la lógica de la heurística, el bucle de aceptación por umbrales, genera los lotes, guarda y modifica el estado de la temperatura y permite hacer barridos.

4.2 Flujo de ejecución

1. Recolecta los parámetros del sistema, se carga cada ciudad y conexiones de la base de datos en memoria.
2. Para cada corrida que se haya parametrizado se genera un hilo el cual se encarga de:
 - (a) se construye un objeto `Tour`: se filtran las conexiones relevantes a la instancia suplida, se calcula $d_{\max}$, $N(I)$ y la matriz de peso, se permuta la solución inicial.
 - (b) se corre `accept_solutions`.
 - (c) si la bandera `-activate-sweep` fue activada, se corre el barrido.
 - (d) se escribe el reporte a disco.

5 Resultados

5.1 Setup

Dos instancias del problema fueron resueltas: una con 40 ciudades (`input-40.tsp`) y otra con 150 ciudades (`input-150.tsp`). Para la instancia de 40 ciudades se corrieron 10,000 semillas, mientras que para la de 150 ciudades se corrieron aproximadamente 100,000 semillas. La elección de los parámetros de temperatura inicial, factor de enfriamiento y tamaño de lote siguió las recomendaciones prácticas descritas por Minaiev [7], quien discute cómo calibrar estos valores de manera experimental en vez de derivarlos analíticamente. Los parámetros usados se desglosan en la siguiente tabla

Table 1: Parámetros				
Instance	T_0	ϕ	L	ε
40 cities	50,000	0.99	500	0.000001
150 cities	500,000	0.995	4000	0.000000001

5.2 Mejores soluciones

Table 2: Mejor costo por instancia			
Instancia	Corridas	Evaluacion	Semilla
40 cities	10,000	0.263819525596896	11818631877859855109
150 cities	100,000	0.136506050427962	3533037625083682721

5.3 Mapas

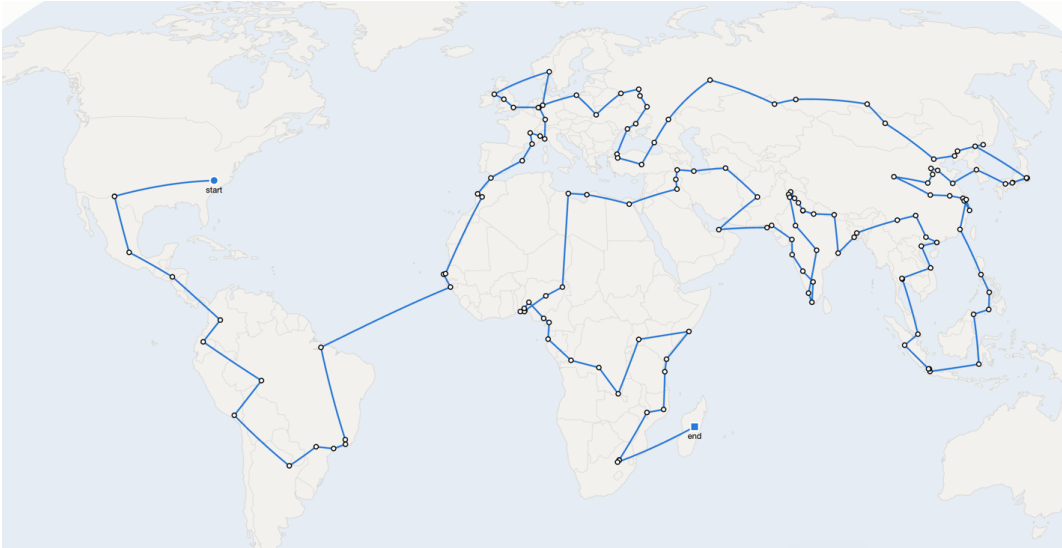


Figure 1: Mejor recorrido – 150 ciudades



Figure 2: Mejor recorrido – 40 ciudades

5.4 Convergencia

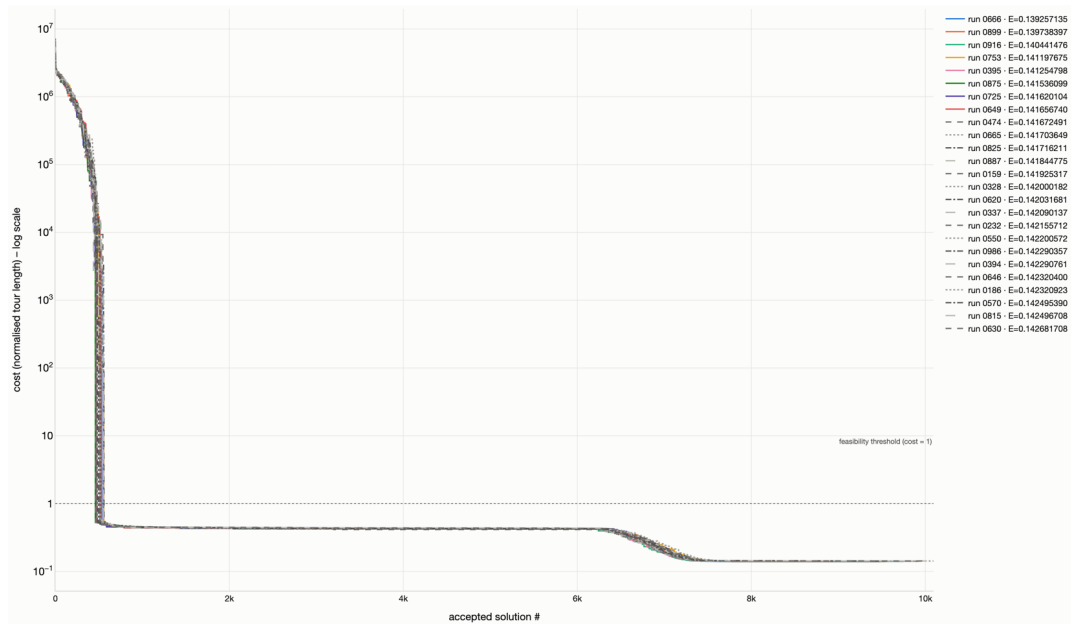


Figure 3: Evaluación de las mejores 25 semillas – Swap (150 ciudades)

Para 150 ciudades, usamos un segundo método de obtener vecinos, el cual es 2-opt, introducido por Croes [2] y popularizado como parte de esquemas de búsqueda local más elaborados por Lin y Kernighan [3]:

modificaremos cuatro aristas del camino y con frecuencia deja cruces en la ruta. El movimiento que hace 2-opt ofrece una alternativa con mejor comportamiento: selecciona dos aristas no adyacentes (s_i, s_{i+1}) y (s_j, s_{j+1}) con $i < j$, las elimina y reconecta el camino como (s_i, s_j) y (s_{i+1}, s_{j+1}) . Sobre la permutación esto equivale a invertir el segmento $s_{i+1}, \dots, s_j$:

$$(\dots, s_i, \underbrace{s_{i+1}, \dots, s_j}_{\text{invertido}}, s_{j+1}, \dots) \longrightarrow (\dots, s_i, s_j, \dots, s_{i+1}, s_{j+1}, \dots). \quad (8)$$

Dado esto, corrimos 10 semillas, obteniendo muchos mejores resultados:

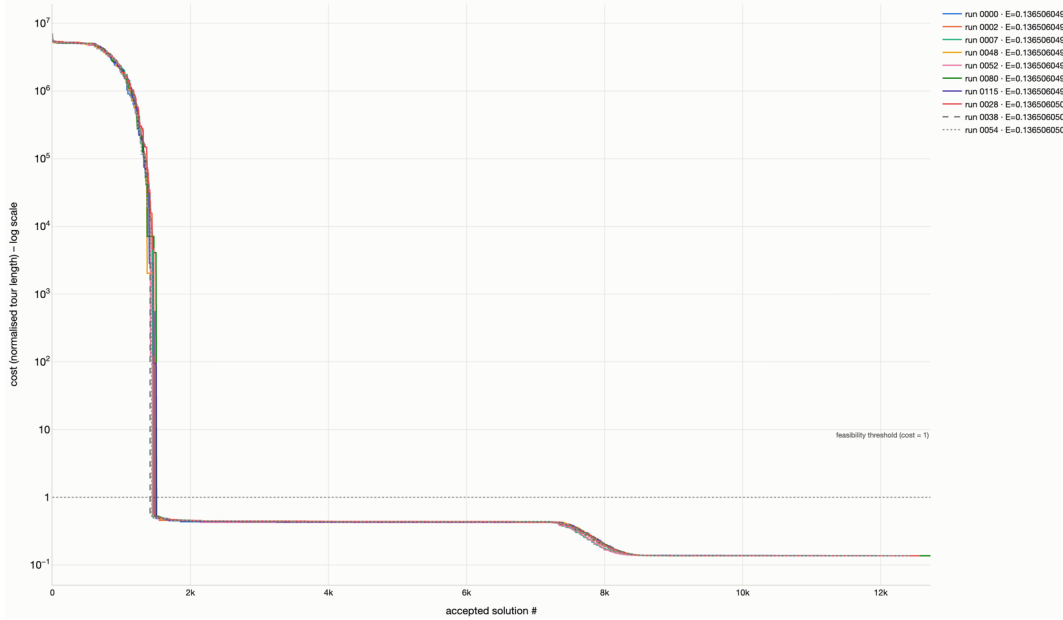


Figure 4: Evaluación de las mejores 10 semillas – 2-opt (150 ciudades)

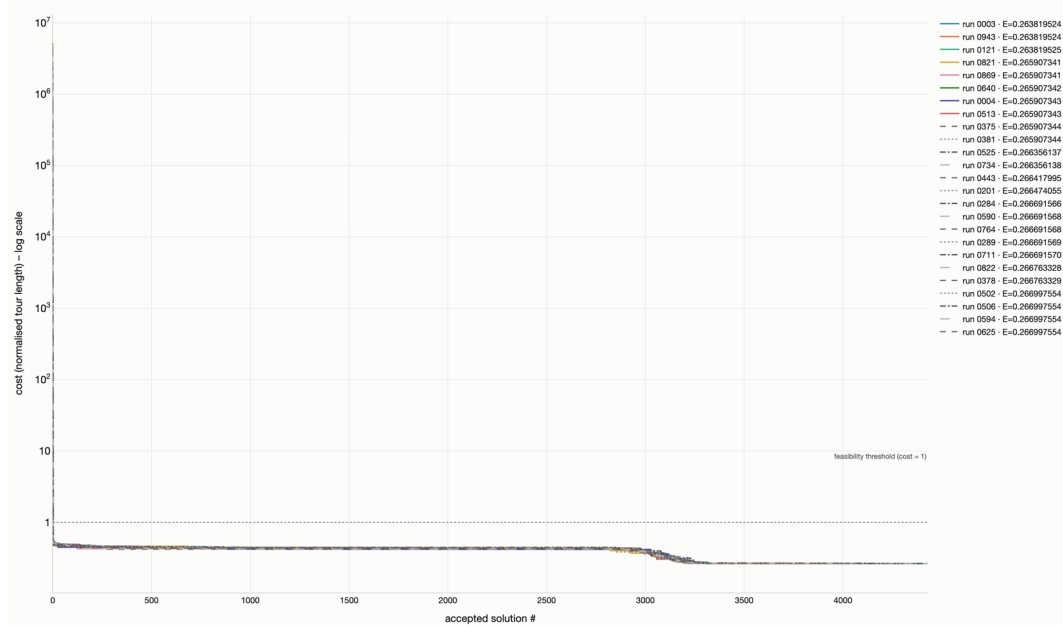


Figure 5: Evaluación de las mejores 25 semillas (40 ciudades)

6 Conclusión

Si bien el modelado e implementación de código fue parte del reto de este proyecto, creo que la parte más interesante fue el momento donde se empezó a experimentar, dado que depende de muchas variables los posibles resultados a encontrar (entre muchos la suerte y vender el alma) lo cual convierte esta segunda etapa en una nueva búsqueda, la cual ya no se centra en construir, sino en explorar nuestras propias implementaciones y descubrir donde fallan y como mejorarlas. De igual manera creo que una forma de mejorar para futuros proyectos y experimentos es la creación de un pequeño framework de automatización de resultados, donde podamos comparar diferentes parámetros del experimento y con ello automatizar estas mejoras (en una especie de metametaheurística).

References

- [1] G. B. Dantzig, D. R. Fulkerson, and S. M. Johnson. Solution of a large-scale traveling-salesman problem. *Journal of the Operations Research Society of America*, 2(4):393–410, 1954.
- [2] G. A. Croes. A method for solving traveling-salesman problems. *Operations Research*, 6(6):791–812, 1958.
- [3] S. Lin and B. W. Kernighan. An effective heuristic algorithm for the traveling-salesman problem. *Operations Research*, 21(2):498–516, 1973.
- [4] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.
- [5] V. Černý. Thermodynamical approach to the traveling salesman problem: An efficient simulation algorithm. *Journal of Optimization Theory and Applications*, 45(1):41–51, 1985.

- [6] G. Dueck and T. Scheuer. Threshold accepting: A general purpose optimization algorithm appearing superior to simulated annealing. *Journal of Computational Physics*, 90(1):161–175, 1990.
- [7] B. Minaiev. Converging on simulated annealing. <https://bminaiev.github.io/simulated-annealing>, January 2023. Accessed September 2026.